ESPECIFICACIÓN DE ARQUITECTURA FRONTEND v2.0

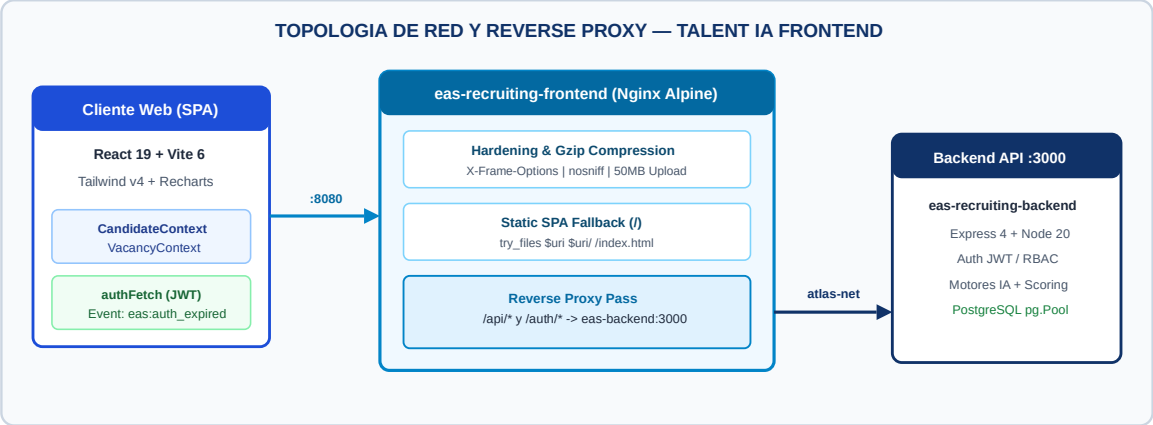
ARQUITECTURA TÉCNICA: TALENT IA FRONTEND

Single Page Application React 19 + Vite 6 + Tailwind CSS v4 + Nginx Alpine

Repositorio:	eaguilarh01/ talent_ia_frontend.git	**Autor:**	Enrique Aguilar
Organización:	EAS Consulting Group	**Fecha:**	Noviembre 2026
Stack UI:	React 19, Vite 6, Tailwind v4	**Contenedor:**	eas-recruiting-frontend (:8080)

1. Visión General y Topología de Red

El repositorio **talent_ia_frontend** encapsula la aplicación cliente SPA de Talent IA. En producción se construye mediante un `Dockerfile` multi-etapa (compilación estática en `node:20-slim` y entrega sobre `nginx:alpine`). La aplicación no realiza conexiones directas hacia PostgreSQL ni almacena llaves maestras de proveedores IA; toda operación protegida transita mediante proxy reverso interno hacia `eas-backend:3000`.



2. Stack Tecnológico del Frontend

Componente	Tecnología	Versión	Función Principal
Librería UI	React	19.0.1	Renderizado reactivo modular, hooks y concurrent mode
Lenguaje	TypeScript	~5.8.2	Contratos de tipos estrictos alineados con el backend
Bundler	Vite	^6.2.3	Servidor dev HMR con proxy /api y build minificado
Estilos	Tailwind CSS v4	^4.1.14	Plugin @tailwindcss/vite, tokens corporativos y media print
Gráficos y UI	Recharts / Lucide / Motion	3.8 / 0.546 / 12.23	Tableros ejecutivos, FitGauge e iconografía SVG
Servidor Web	Nginx Alpine	nginx:alpine	Reverse proxy, compresión Gzip y hardening OWASP

3. Estructura de Directorios y Módulos

```
talent_ia_frontend/

|-- public/                     # eas_logo.png, favicon.ico, favicon.png

|-- src/

|   |-- App.tsx                 # Orquestador principal, router de tabs y login JWT

|   |-- api.ts                  # Cliente HTTP authFetch con inyección Bearer y manejo 401

|   |-- index.css               # Directivas Tailwind v4 y variables corporativas

|   |-- types.ts                # Interfaces Candidate, Vacancy, UserRole

|   |-- components/             # AdminPanel, AuditLogPanel, CalendarPanel, CandidatePanel,

|   |                           # CareersPortal, DashboardPanel, Sidebar, VacanciesPanel, Modales

|   |-- context/                # CandidateContext.tsx, VacancyContext.tsx

|   -- utils/                  # canonicalCvNormalizer, doubleCheckEvaluator, questionEngine,

|                               # vacancyUtils, textRepair y suites *.test.ts (tsx)

|-- docs/                       # PDFs oficiales y diagrama_arquitectura.drawio

|-- Dockerfile                  # Build multi-etapa Node 20 -> Nginx Alpine

|-- nginx.conf                  # Gzip, hardening HTTP, SPA fallback y proxy /api/*

-- package.json                # Scripts dev, build, preview, lint, test
```

4. Seguridad HTTP, Proxy Reverso Nginx y Sesión JWT

4.1 Hardening y Proxy Reverso (nginx.conf)

El archivo `nginx.conf` habilita compresión Gzip, limita el tamaño máximo de carga de CVs a `50M`, aplica cabeceras de seguridad (`X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`) y enruta `/api/*` y `/auth/*` hacia `http://eas-backend:3000`.

4.2 Interceptor authFetch y Cierre Automático

El cliente centralizado en `src/api.ts` lee el token JWT desde `localStorage` y lo adjunta en la cabecera `Authorization: Bearer <token>`. Ante una respuesta HTTP `401 Unauthorized` emite el evento global `eas:auth_expired` para retornar limpiamente a la pantalla de autenticación. Se descartó cualquier integración con Microsoft M365.

4.3 Visibilidad por Rol RBAC

- **ADMIN:** Acceso completo incluyendo gestión de usuarios, auditoría inmutable y controles ISO/LFPDPPP.

- **RECRUITER:** Operación integral del pipeline (vacantes, carga de CVs, FitScore, Double-Check, speeches y calendario).
- **MANAGER:** Consulta de vacantes asignadas, scorecard de entrevistas, revisión Double-Check y aprobación.

5. Comandos Operativos

```
npm install          # Instalar dependencias

npm run dev          # Desarrollo local (Vite :5173 con proxy hacia :3000)

npm run lint         # Verificación estricta tsc --noEmit

npm test             # Ejecución de pruebas unitarias tsx

npm run build        # Compilación productiva en dist/
```